Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1

«ПРИНЦИПЫ РАЗРАБОТКИ ОПЕРАЦИОННЫХ СИСТЕМ»

по дисциплине «Операционные системы»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Ассистент Гавва Г.Д

<подпись>

Санкт-Петербург
2026

Содержание

Цель работы 3
Задачи работы 3-4
Ход работы 4-29
1. Теоретическая часть 4-6
1.1 Принцип хранения информации на магнитных носителях 4-5
1.2 Процесс загрузки компьютера 5-6
1.3 Исполнение загрузочного кода 6
1.4 Основные функции BIOS 6-7
1.5 Системы защиты современных процессоров 7-8
1.6 Системы защиты современных процессоров 8-9
1.7 Основы архитектуры IA-32 9
1.8 Защищённый режим IA-32 9-10
1.9 Таблицы дескрипторов 10-12
2. Практическая часть 13-30
2.1 Загрузчик ОС 13-17
2.2 Ядро ОС 17-30
Тестирование ядра ОС 30-31
Вывод 32
Листинг А «bootsect.asm» 33-36
Листинг Б «kernel.cpp» 37-66

ЦЕЛЬ РАБОТЫ

Цель работы - изучение основ разработки ОС, принципов низкоуровневого взаимодействия с аппаратным обеспечением, программирования системной функциональности и процесса загрузки системы.

ЗАДАЧИ РАБОТЫ:

1) Изучить теоретические основы разработки операционных систем, а именно: процесс загрузки персонального компьютера, структуру и назначение загрузочного сектора, а также принципы функционирования процессора IA-32 в реальном и защищенном режимах. Особое внимание уделить пониманию роли BIOS, системных прерываний и механизмов защиты памяти.

2) Освоить инструментарий разработки: транслятор ассемблера GNU as (с синтаксисом Intel) для создания загрузчика и компилятор g++ для компиляции ядра на языке C++, настройка процесса компиляции и линковки для получения бинарных образов, а также запуск и отладка полученной системы в эмуляторе QEMU.

3) Разработать загрузчик, соответствующий варианту CalcOS. Его функции: обеспечение взаимодействия с пользователем для выбора цвета текста (с выводом меню и обработкой нажатий клавиш '1'-'6'), сохранение выбранного параметра в память для последующей передачи ядру, корректное считывание образа ядра со второго гибкого диска в оперативную память по адресу 0x10000, а также выполнение всех необходимых действий для перевода процессора в защищенный 32-битный режим (отключение прерываний, включение линии A20, загрузка GDT, установка бита PE) и передача управления ядру.

4) Реализовать минимальное ядро операционной системы на языке C++. Его задачи: считывание параметра цвета, переданного загрузчиком; настройка системы прерываний (заполнение таблицы IDT, загрузка ее в процессор, регистрация обработчика клавиатуры); организация

интерактивного пользовательского интерфейса с поддержкой ввода команд, редактирования (клавиша Backspace) и ограничением длины строки; реализация команд info (вывод информации о системе), shutdown (выключение) и, ключевой команды expr, которая должна представлять собой полноценный калькулятор, способный разбирать арифметические выражения с учетом приоритетов операций и корректно обрабатывать ошибки, такие как некорректное выражение, деление на ноль и целочисленное переполнение.

5) Комплексная проверка разработанной системы. Требовалось составить набор из не менее чем 15 тестов, охватывающих все реализованные команды и возможные ошибочные ситуации, и провести тестирование, чтобы убедиться в стабильности и корректности работы загрузчика и ядра.

ХОД РАБОТЫ

1. Теоретическая часть

1.1 Принцип хранения информации на магнитных носителях

Хранение информации на магнитных носителях базируется на трёх ключевых понятиях адресации данных: **головка**, **цилиндр** и **сектор**. В многослойных носителях (например, жёстких дисках) каждый физический диск обслуживается собственной магнитной головкой, номер которой соответствует номеру диска в пакете. Поверхность каждого диска разделена на концентрические кольцеобразные дорожки, объединённые по вертикали в цилиндры. Каждая дорожка, в свою очередь, разбита на секторы фиксированного размера — традиционно 512 байт. Операции чтения и записи выполняются минимально по одному сектору, для чего необходимо указать тройку координат: номер головки, цилиндра (дорожки) и сектора.

Наглядным примером является гибкий магнитный диск (дискета), имеющий следующую геометрию: 2 головки, 80 цилиндров на головку и 18 секторов на цилиндр, что даёт общее количество 2 880 секторов. Нумерация элементов осуществляется следующим образом: головки и цилиндры нумеруются с нуля, секторы — с единицы. Особый статус имеет первый

сектор диска с адресом 0:0:1 (головка 0, цилиндр 0, сектор 1) — загрузочный сектор, содержащий 512-байтный код, выполняемый BIOS при старте системы. Порядок нумерации секторов выбран оптимальным образом: при последовательном увеличении логического номера сектора сначала инкрементируется номер головки, затем — номер цилиндра. Такая организация минимизирует механические перемещения блока головок при чтении последовательных данных, повышая производительность операций ввода-вывода. Для преобразования абсолютного номера сектора в геометрические координаты (головка, цилиндр, сектор) применяются специальные формулы, учитывающие геометрию носителя.

1.2 Процесс загрузки компьютера

Процесс загрузки компьютера представляет собой последовательность этапов, начинающуюся с момента включения питания и завершающуюся передачей управления операционной системе. После подачи питания процессор выполняет встроенное самотестирование (**BIST** — Built-In Self Test), после чего начинает выполнение первой инструкции по фиксированному физическому адресу 0xFFFFFFF0, где расположен код постоянного запоминающего устройства (**EPROM**). Эта программа представляет собой базовую систему ввода-вывода (**BIOS** — Basic Input/Output System), задача которой — инициализировать аппаратное обеспечение, провести диагностику его работоспособности и определить устройство, с которого будет выполняться загрузка операционной системы. В соответствии с настройками BIOS поиск загрузчика осуществляется на различных носителях: гибких дисках, жёстких дисках, оптических приводах или сетевых ресурсах. При обнаружении загрузочного носителя BIOS считывает его **первый сектор (512 байт)**, называемый **загрузочным сектором**, в оперативную память по адресу 0x00007C00. Носитель считается загрузочным, если по смещению 510 байт от начала сектора расположена сигнатура 0x55AA. При успешной проверке BIOS передаёт управление загруженному коду, который начинает выполняться с адреса **0x00007C00**. В

этот момент процессор работает в реальном режиме (режим совместимости с архитектурой Intel 8086), что накладывает ограничения на адресуемую память (около 1 МБ). Основная задача загрузочного кода — подготовить среду для работы операционной системы: инициализировать таблицы дескрипторов (GDT), включить линию адреса A20, перевести процессор в защищённый 32-битный режим путём установки бита PE регистра CR0, а также, при необходимости, загрузить ядро ОС из последующих секторов носителя в оперативную память и передать ему управление.

1.3 Исполнение загрузочного кода

После передачи управления загрузочному сектору (по адресу 0x00007C00) процессор архитектуры IA-32 (x86) работает в реальном режиме — режиме обратной совместимости с процессором Intel 8086. В этом режиме все инструкции являются 16-битными, память адресуется с помощью сегментной модели: физический адрес вычисляется как *сегмент* × *16* + *смещение*. Например, обращение по логическому адресу 0400h:00F0h преобразуется в физический адрес 0400h × 16 + 00F0h = 40F0h. Такая схема ограничивает адресуемое пространство примерно 1 МБ (2^{20} байт). Загрузочный код, размещённый в первых 512 байтах диска, должен уложиться в эти ограничения и завершаться сигнатурой 0x55AA для подтверждения загрузочного статуса носителя. Типичный загрузчик инициализирует сегментные регистры (DS, SS), устанавливает стек, использует функции BIOS через прерывания (например, int 0x10 для вывода символов, int 0x13 для чтения секторов с диска) и готовит переход в защищённый режим. После выполнения подготовительных операций загрузчик считывает ядро операционной системы с диска в оперативную память и передаёт ему управление.

1.4 Основные функции BIOS

BIOS (Basic Input/Output System — базовая система ввода-вывода) предоставляет программному обеспечению стандартизированный интерфейс для взаимодействия с аппаратным обеспечением через механизм

прерываний. Для вызова функции BIOS данные помещаются в определённые регистры процессора, после чего выполняется команда int с указанием номера прерывания; номер конкретной функции внутри прерывания задаётся в регистре AH. Ключевые функции, используемые при разработке загрузчика, включают: видеосервис int 0x10, позволяющий устанавливать текстовый режим терминала (функция 0x00) или выводить отдельные символы на экран в режиме эмуляции телетайпа (функция 0x0E, где символ передаётся в регистре AL); дисковый ввод-вывод int 0x13, функция 0x02 которого осуществляет чтение секторов с носителя в оперативную память (параметры: DL — номер диска, DH — номер головки, CH — номер цилиндра, CL — номер сектора, AL — количество секторов, ES:BX — адрес буфера в памяти; при ошибке устанавливается флаг CF); а также сервис клавиатуры int 0x16, функция 0x00 которого приостанавливает выполнение программы до нажатия клавиши пользователем и возвращает её скан-код. Эти функции образуют фундаментальную «прослойку» между низкоуровневым кодом загрузчика и физическим оборудованием, позволяя разработчику операционной системы абстрагироваться от аппаратных различий конкретных моделей компьютеров и сосредоточиться на логике загрузки.

1.5 Системы защиты современных процессоров

Современные процессоры архитектуры x86 реализуют многоуровневую систему защиты, основанную на концепции колец привилегий (protection rings) и аппаратной изоляции ресурсов. В защищённом режиме процессор поддерживает четыре уровня привилегий (0–3), где кольцо 0 обладает максимальными правами (полный доступ к аппаратуре, привилегированные инструкции), а кольцо 3 — минимальными (пользовательские приложения). Привилегии определяются через биты привилегий (DPL — Descriptor Privilege Level) в дескрипторах сегментов глобальной таблицы дескрипторов (GDT) и локальной таблицы дескрипторов (LDT), а также через текущий уровень привилегий процесса (CPL — Current

Privilege Level), хранящийся в сегментном регистре CS. Доступ к сегменту разрешён только при выполнении условия $CPL \leq DPL$; иначе возникает исключение защиты общего назначения (#GP). Критически важные инструкции (например, LGDT, LIDT, HLT, изменение регистра CR0) помечены как привилегированные и могут выполняться исключительно в кольце 0. Для безопасного перехода между уровнями привилегий используются шлюзы вызовов (call gates) или специальные прерывания: при вызове системной функции процессор автоматически переключает стек на привилегированный уровень, сохраняя контекст вызывающей программы, а при возврате — восстанавливает его через инструкцию IRET. Дополнительный уровень защиты обеспечивает страничная организация памяти: каждая запись в таблице страниц содержит биты присутствия (P), прав доступа (R/W), привилегий (U/S — User/Supervisor) и модификации (D), что позволяет изолировать адресные пространства процессов и предотвратить несанкционированное обращение к памяти ядра. Такая многослойная архитектура гарантирует, что пользовательские приложения не могут напрямую манипулировать аппаратными ресурсами или памятью других процессов, обеспечивая стабильность и безопасность операционной системы.

1.6 Системы защиты современных процессоров

Защита процессора представляет собой набор механизмов, обеспечивающих безопасное выполнение кода за счет контроля доступа к ресурсам системы. Основные функции включают запрет выполнения критических инструкций, ограничение доступа к областям памяти и внешним устройствам. В современных процессорах применяется иерархия уровней привилегий: в архитектуре **Intel IA-32** используется четыре уровня (кольца 0–3), где уровень 0 обладает максимальными правами **(Супервизор)**, а уровень 3 — минимальными **(Пользователь)**. На практике большинство операционных систем используют упрощенную модель с двумя уровнями: ядро ОС работает на уровне 0, имея полный контроль над системой, а

пользовательские приложения — на уровне 3, ограниченные в операциях. Для безопасного взаимодействия с ядром применяется механизм системных вызовов: при выполнении запроса (например, открытия файла) процессор временно переключается в режим **Супервизора**, позволяя выполнить привилегированную операцию, после чего права возвращаются к пользовательскому уровню. Это гарантирует изоляцию приложений друг от друга и защиту критических системных компонентов от несанкционированного доступа.

1.7 Основы архитектуры IA-32

Реальный режим работы процессора предназначен для совместимости с предыдущими моделями процессоров и механизмы защиты в нем отсутствуют. **Защищенный режим** — основной режим процессора, который должны использовать все современные ОС, и который позволяет использовать все возможности процессора (32-битную адресацию памяти, все инструкции, аппаратную многозадачность, защиту). Задача защищенного режима может выполняться в режиме виртуального 8086, который очень похож на реальный режим. Таким образом, например, осуществляется эмуляция MS-DOS в системе Windows. При включении компьютера процессор оказывается в реальном режиме и задача любой программы-загрузчика ОС — перевести процессор в защищенный режим.

1.8 Защищённый режим IA-32

Защищенный режим архитектуры IA-32 использует сегментацию памяти для обеспечения безопасности. Каждый сегмент описывается дескриптором (8-байтная структура), который содержит:

- Базовый адрес (начало сегмента в памяти),
- Лимит (размер сегмента),
- Тип (код или данные),
- Уровень привилегий (0–3, где 0 — максимальные права, 3 — минимальные).

При обращении к памяти процессор проверяет дескриптор соответствующего сегмента. Например, ядро ОС размещается в сегменте уровня 0, а пользовательские программы — в сегменте уровня 3. Это предотвращает несанкционированный доступ: программа не может изменить данные ядра или других приложений, даже если знает их адреса. Такой механизм гарантирует стабильность системы и защиту критических ресурсов.

1.9 Таблицы дескрипторов

Таблица дескрипторов — это структура данных в защищенном режиме процессора, которая содержит описание сегментов памяти и других системных ресурсов. Она используется для реализации сегментации и защиты памяти в современных операционных системах.

В архитектуре IA-32 существует два основных типа таблиц дескрипторов: **глобальная таблица дескрипторов (GDT)**, которая содержит дескрипторы, доступные для всей системы, используется для описания системных сегментов (ядра ОС, системных ресурсов), единая для всех программ и процессов в системе, адрес GDT хранится в регистре GDTR ; **локальная таблица дескрипторов (LDT)**, которая содержит дескрипторы, доступные только для конкретной программы, используется для описания сегментов, принадлежащих одной программе, каждая программа может иметь свою собственную LDT, адрес LDT хранится в сегментном регистре.

Дескриптор имеет размер **8 байт** и содержит следующую информацию:

Основные поля дескриптора:

1. **База (Base)** — 32-битный адрес начала сегмента
2. **Лимит(Limit)** — 20-битный размер сегмента (может масштабироваться до 4 ГБ при использовании бита G)
3. **Тип сегмента** — определяет, является ли сегмент кодом или данными, а также права доступа
4. **Уровень привилегий (DPL)** — определяет, какие программы могут использовать этот сегмент

5. **Бит присутствия (P)** — указывает, находится ли сегмент в памяти (1) или выгружен на диск (0)
6. **Бит гранулярности (G)** — определяет масштаб лимита (0 = 1 байт, 1 = 4 КБ)
7. **Бит размера (B)** — определяет размерность сегмента (0 = 16-битный, 1 = 32-битный)

Сегментные регистры и селекторы

В защищенном режиме сегментные регистры имеют размер **10 байт.** Из них **2 байта доступны для программиста** и называются **селектором. 8 байт недоступны для пользователя** и называются **теневой частью (кэшем дескриптора)**

Структура селектора (2 байта):

- **Биты 16-3** — номер дескриптора в таблице
- **Бит 2** — указывает на используемую таблицу (0 = GDT, 1 = LDT)
- **Биты 1-0** — запрашиваемый уровень привилегий (RPL)

При загрузке селектора в сегментный регистр, процессор автоматически загружает соответствующий дескриптор в теневую часть регистра, что ускоряет доступ к сегментам.

Для работы в защищенном режиме необходима минимальная таблица, содержащая как минимум три записи:

1. **Нулевой дескриптор** — заполненный нулями, используется как "пустой" дескриптор
2. **Дескриптор кода** — описывает область памяти, где находится программный код
3. **Дескриптор данных** — описывает область памяти, где находятся данные

Кроме GDT и LDT, существует еще одна важная таблица — **IDT (Interrupt Descriptor Table)**, которая используется для обработки прерываний и исключений: она содержит дескрипторы обработчиков

прерываний, каждый дескриптор описывает адрес обработчика прерывания и его параметры, адрес IDT хранится в регистре IDTR, формат дескриптора IDT включает адрес обработчика, селектор сегмента и тип шлюза

Таблицы дескрипторов позволяют реализовать:

Защиту памяти — предотвращение несанкционированного доступа к областям памяти

Сегментацию — разделение памяти на логические сегменты (код, данные, стек)

Многозадачность — изоляция задач друг от друга

Управление прерываниями — обработка аппаратных и программных прерываний

2. Практическая часть

2.1 Загрузчик ОС

Загрузчик разработан на языке ассемблера с использованием синтаксиса Intel для транслятора GNU as.

При получении управления процессор находится в реальном режиме. Первым делом загрузчик выполняет инициализацию сегментных регистров:

```
_start:
        # Инициализация сегментов
        mov ax, cs
        mov ds, ax
        mov ss, ax
        mov sp, 0x7C00
```

Регистр CS уже указывает на сегмент, содержащий наш код. Эти инструкции копируют его значение в регистры данных DS и стека SS, а указатель стека SP устанавливается на начало загрузочного сектора. Стек будет расти вниз от адреса 0x7C00, не перекрывая код.

Для обеспечения корректного отображения текста на экране вызывается функция BIOS int 0x10 с номером функции 0x00, которая устанавливает текстовый режим 80x25 символов:

```
mov al, 0x03
mov ah, 0x00
int 0x10
```

Загрузчик выводит на экран меню с помощью функции puts, которая использует сервис BIOS int 0x10, ah=0x0E (телетайпный вывод):

```
 # Вывод сообщения (Используем BIOS int 0x10)
 mov bx, offset loading_str
 call puts
loading_str:
        .asciz "Press: 1-green, 2-blue, 3-red, 4-yellow, 5-gray, 6-white\r\n"
```

Далее загрузчик ожидает нажатия клавиши пользователем. Для этого используется функция BIOS int 0x16, ah=0x00, которая ожидает нажатия и возвращает скан-код и ASCII-символ нажатой клавиши в регистре al:

```
get_choice:
        xor ah, ah
        int 0x16

        cmp al, '1'
        jb get_choice
        cmp al, '6'
        ja get_choice

        sub al, '1'
        mov bx, offset color_table
        xlat

        # Сохраним в 0x8000
        push ax
        xor ax, ax
        mov es, ax
        pop ax
        mov es:[0x8000], al

    mov al, 0x03
        mov ah, 0x00
        int 0x10
```

После получения символа выполняется проверка, что нажата цифра от '1' до '6'. Если нажата другая клавиша, ввод игнорируется:

Выбранный пользователем символ (от '1' до '6') преобразуется в код атрибута цвета для текстового режима VGA. Это делается с помощью таблицы перекодировки color_table и инструкции xlat:

Таблица color_table содержит байты атрибутов цвета:

```
color_table:
        .byte 0x02, 0x01, 0x04, 0x0E, 0x07, 0x0F
        #    green  blue  red  yellow gray  white
```

Полученный байт цвета сохраняется в память по заранее определенному адресу 0x8000. Этот адрес был выбран произвольно, как область памяти, гарантированно не занятая загрузчиком (0x7C00-0x7DFF) и не конфликтующая с областью данных BIOS (0x400-0x500) или будущим расположением ядра (0x10000):

Перед загрузкой ядра экран очищается повторной установкой видеорежима. Затем с помощью функции BIOS int 0x13, ah=0x02 (чтение секторов с диска) загрузчик считывает образ ядра со второго гибкого диска:

```
# Загрузка ядра со второй дискеты (fd1)
    # Читаем секторы в адрес 0x10000 0x10000:0x0000)
    mov bx, 0x1000 # Сегмент ES:BX = 0x1000:0x0000 -> физ. адре 0x10000
    mov es, bx
    xor bx, bx
    mov dl, 0x01   # Disk B
    mov dh, 0x00   # Номер головки
    mov ch, 0x00   # Цилиндр 0
    mov cl, 0x01   # Сектор 1
    mov al, 0x1A   # Кол-во секторов к чтению
    mov ah, 0x02   # Код функции чтения секторов
    int 0x13       # Вызов BIOS
```

Загрузка производится по физическому адресу 0x10000 (сегмент 0x1000, смещение 0x0000), что соответствует точке сборки ядра, указанной при линковке (-Ttext 0x10000). Чтение 26 секторов (по 512 байт) позволяет загрузить ядро размером до 13 КБ, что достаточно для разработанной функциональности.

Дальше происходит переключение процессора в защищенный режим:

```
# Переход в защищённый режим (упрощённо)
cli               # Отключить прерывания

in al, 0x92
or al, 2
out 0x92, al

lgdt gdt_info   # Загрузить таблицу дескрипторов

mov eax, cr0
or al, 1          # Включить бит PE
mov cr0, eax

jmp 0x8:protected_mode
```

Команда cli запрещает обработку маскируемых прерываний, чтобы процессор не отвлекся во время переключения режимов.

В старых системах линия A20 была отключена для эмуляции поведения 8086. Для доступа ко всей памяти выше 1 МБ ее необходимо включить. Это делается через системный порт 0x92.

Загрузка глобальной таблицы дескрипторов (GDT): GDT — это структура данных, которая определяет сегменты памяти в защищенном режиме. Команда lgdt загружает в процессор указатель на эту таблицу (ее адрес и размер):

```
gdt:
        .byte 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00  # Null descriptor
        .byte 0xff, 0xff, 0x00, 0x00, 0x00, 0x9A, 0xCF, 0x00  # Code segment(0x08)
        .byte 0xff, 0xff, 0x00, 0x00, 0x00, 0x92, 0xCF, 0x00  # Data segment(0x10)
gdt_end:
```

Используемая GDT содержит три 8-байтных дескриптора:

Нулевой дескриптор: Обязателен и не используется процессором.

Сегмент кода (селектор 0x08): Описывает 32-битный сегмент кода с базой 0, лимитом 4 ГБ, уровнем привилегий 0 (самый привилегированный).

Сегмент данных (селектор 0x10): Описывает 32-битный сегмент данных с базой 0, лимитом 4 ГБ, уровнем привилегий 0.

Включение бита защиты (PE): Установка младшего бита в регистре управления CR0 переводит процессор в защищенный режим:

```
mov eax, cr0
or al, 1          # Включить бит PE
mov cr0, eax

jmp 0x8:protected_mode
```

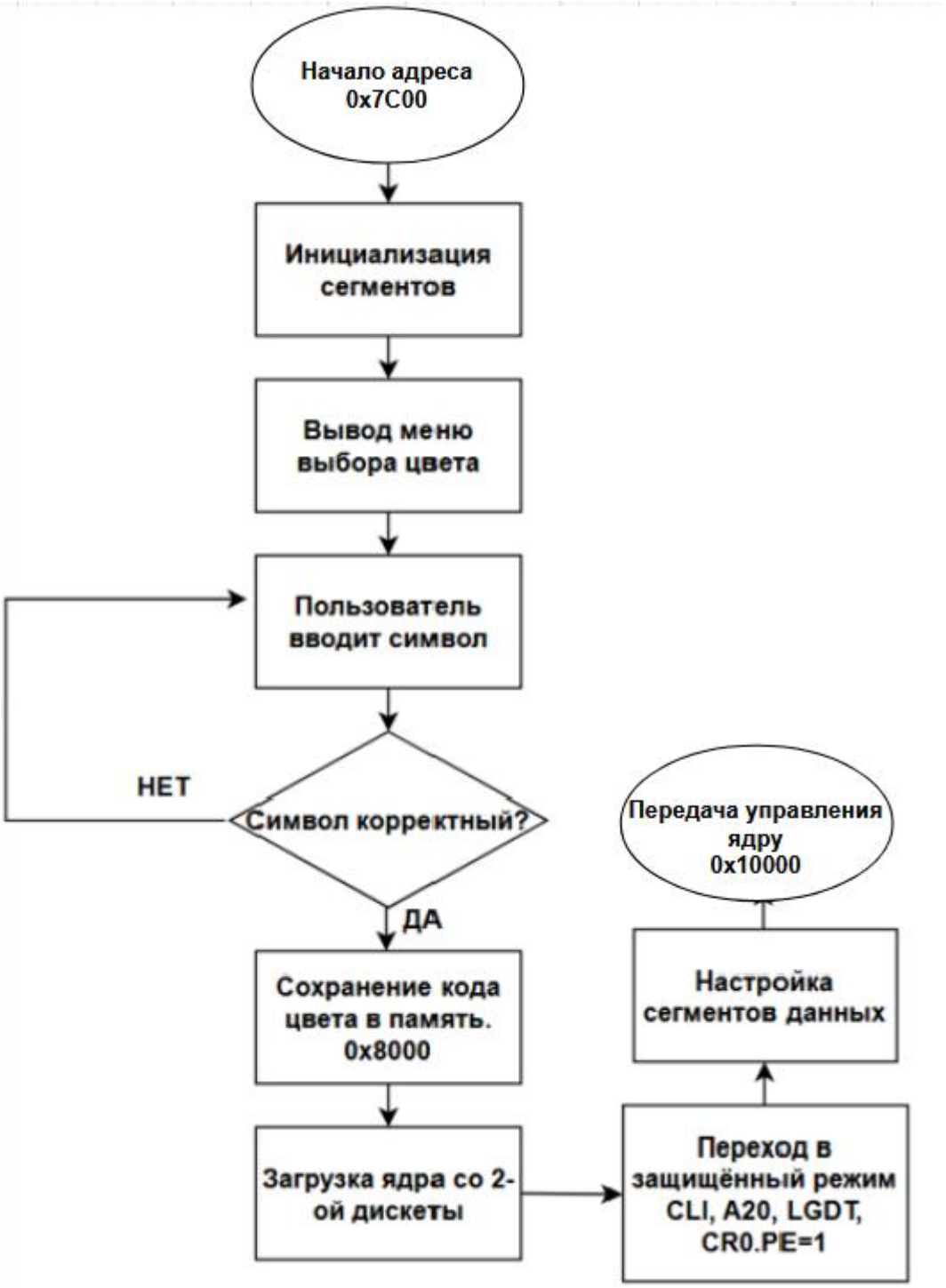
После включения защищенного режима необходимо выполнить "дальний" прыжок, чтобы загрузить новый селектор в регистр CS и сбросить очередь предвыборки команд.

Здесь 0x8 — это селектор, указывающий на второй дескриптор в GDT (сегмент кода). Метка protected_mode находится в этом же файле, но после перехода процессор начинает интерпретировать код как 32-битный.

После входа в защищенный режим (метка protected_mode) необходимо настроить сегментные регистры данных на новый селектор данных (0x10) и установить указатель стека на безопасную область памяти:

```
.code32
protected_mode:
        # Настройка сегментов данных
        mov ax, 0x10
        mov ds, ax
        mov es, ax
        mov ss, ax
        mov esp, 0x90000
        call 0x10000
```

Инструкция call 0x10000 выполняет вызов кода по физическому адресу, куда ранее было загружено ядро. Управление окончательно передается от загрузчика к ядру операционной системы.



Блок схема алгоритма работы загрузчика

2.2 Ядро ОС

Метод решения задачи взаимодействия с загрузчиком

Согласно варианту задания, загрузчик запрашивает у пользователя цвет текста и сохраняет его в память по адресу 0x8000. Ядро должно считать этот параметр и применить его.

Метод решения: В функции kmain первым же действием выполняется чтение байта по абсолютному адресу 0x8000:

```
current color = *(volatile unsigned char*)0x8000;
```

Использование квалификатора volatile необходимо, чтобы компилятор не оптимизировал это обращение к памяти. Полученное значение цвета затем используется во всех функциях вывода на экран (screen_put_char, screen_print_string), а также при выводе команды info для отображения выбранного пользователем цвета в текстовом виде.

Метод решения задачи настройки прерываний и обработки клавиатуры

Для обеспечения интерактивного ввода команд необходимо настроить систему прерываний и реализовать обработчик клавиатуры.

Метод решения:

Создание структур данных: Были определены структуры InterruptDescriptor (для записи в IDT) и DescriptorPointer (для команды lidt), точно соответствующие форматам, требуемым процессором Intel.

```
struct InterruptDescriptor {
    unsigned short offset_low;
    unsigned short segment;
    unsigned char reserved;
    unsigned char attributes;
    unsigned short offset_high;
} __attribute__((packed));

struct DescriptorPointer {
    unsigned short table_size;
    unsigned int table_base;
} __attribute__((packed));
```

Инициализация IDT: Функция initialize_interrupts заполняет все 256 записей таблицы IDT обработчиком по умолчанию (default_handler), который

просто выполняет возврат из прерывания. Это гарантирует, что любое неожиданное прерывание не приведет к зависанию системы.

```
static void initialize_interrupts()
{
    int total_entries = sizeof(idt_entries) / sizeof(idt_entries[0]);

    for (int vector_idx = 0; vector_idx < total_entries; vector_idx++) {
        register_interrupt(vector_idx, CODE_SELECTOR, 0x80 | GATE_TYPE, default_handler);
    }
}
```

Регистрация обработчика клавиатуры: Функция setup_keyboard перезаписывает запись для вектора 0x09 (аппаратное прерывание клавиатуры) адресом функции keyboard_handler. Также она размаскирует соответствующее прерывание в контроллере PIC (Programmable Interrupt Controller), записывая маску в порт 0x21:

```
static void setup_keyboard()
{
    register_interrupt(0x09, CODE_SELECTOR, 0x80 | GATE_TYPE, keyboard_handler);
    port_out_byte(PIC_BASE + 1, 0xFF ^ 0x02);
}
```

Функция keyboard_handler:

Сохраняет состояние регистров процессора (pusha). Проверяет, есть ли данные от клавиатуры (чтение порта 0x64)

Читает скан-код из порта 0x60

Отслеживает состояние клавиши Shift (скан-коды 0x2A, 0x36 — нажатие; 0xAA, 0xB6 — отпускание)

Преобразует скан-код в ASCII-символ с использованием таблиц раскладки (unshifted_layout и shifted_layout)

Если включен ввод (input_enabled == 1), обрабатывает символ:

Обычные символы добавляются в буфер и выводятся на экран

Backspace (\b) удаляет последний символ из буфера и с экрана

Enter (\n) устанавливает флаг command_available = 1, сигнализируя главному циклу о готовности команды

Отправляет сигнал "конец прерывания" (0x20) в контроллер PIC, чтобы разрешить следующие прерывания

Восстанавливает регистры и выполняет возврат из прерывания (iretd)

Метод решения задачи реализации команды info

Команда info должна выводить информацию об авторе, средствах разработки и параметре, переданном от загрузчика.

Метод решения: Функция execute_info последовательно вызывает screen_print_string для вывода каждой строки информации. Для отображения выбранного цвета используется переключатель switch, который преобразует числовой код цвета (например, 0x02) в текстовое описание ("green"). Затем строки конкатенируются (вручную, через копирование символов) и выводятся на экран.

Метод решения задачи реализации команды shutdown

Команда shutdown должна выключать компьютер.

Метод решения: Для выключения в эмуляторе QEMU используется запись значения 0x2000 в порт 0x604. Функция execute_shutdown выводит сообщение "Powering off..." и выполняет эту запись с помощью port_out_word. После этого процессор переводится в бесконечный цикл с инструкцией hlt на случай, если выключение не сработало.

Метод решения задачи реализации команды expr (калькулятор)

Это наиболее сложная часть ядра. Команда expr должна разбирать арифметическое выражение, учитывать приоритет операций, поддерживать унарные операторы, корректно обрабатывать ошибки (некорректное выражение, переполнение, деление на ноль).

Метод решения: Был реализован алгоритм, основанный на **двух стеках** (чисел и операторов), напоминающий алгоритм сортировочной станции (shunting-yard algorithm) Эдсгера Дейкстры.

Стеки: Определены два глобальных массива number_stack[STACK_DEPTH] и operator_stack[STACK_DEPTH] с соответствующими функциями push_number, pop_number, push_operator,

pop_operator. Стеки позволяют хранить промежуточные результаты и операторы в процессе разбора.

Приоритет операций: Для определения приоритета используется матрица precedence_grid[4][4], где строки и столбцы соответствуют операторам (+, -, *, /). Функция has_priority возвращает 1, если приоритет первого оператора выше или равен второму.

Разбор выражения (функция evaluate_math_expression):

Выражение разбирается символ за символом слева направо

Поддерживаются **унарные операторы**: при обнаружении '+' или '-' в начале выражения или после другого оператора они интерпретируются как знак числа (переменная unary накапливает знак)

Числа извлекаются функцией fetch_numeric, которая также проверяет переполнение при преобразовании строки в целое число

При обнаружении **бинарного оператора** (+, -, *, /):

Пока стек операторов не пуст и оператор на вершине стека имеет приоритет над текущим, выполняются операции из стека

Текущий оператор помещается в стек операторов

Выполнение операций: Функция perform_operation вызывает соответствующую функцию (safe_add, safe_sub, safe_mul, safe_div), каждая из которых перед вычислением проверяет возможность переполнения (сравнивая операнды с граничными значениями MAX_INT и MIN_INT). Функция safe_div также проверяет деление на ноль.

Завершение разбора: После достижения конца строки выполняются все оставшиеся операции из стека операторов. Если в стеке чисел в итоге остается ровно одно число — вычисление успешно.

Обработка ошибок:

Код 2 (неверное выражение): возвращается, если не удалось распознать число там, где оно ожидалось, или если после разбора в стеке чисел осталось не одно число

Код 3 (переполнение): возвращается любой из функций safe_* при обнаружении переполнения

Код 4 (деление на ноль): возвращается функцией safe_div

Вывод результата: Функция render_number преобразует целое число в строку для вывода на экран. Особое внимание уделено корректной обработке минимального значения -2147483648, которое нельзя просто превратить в положительное число (произойдет переполнение).

Имя функции	**Аргументы**	**Возвращаемое значение**	**Назначение**
БАЗОВЫЕ ФУНКЦИИ ВВОДА-ВЫВОДА			
screen_put_char	color (unsigned char) — цвет символа row (int) — строка экрана (0-24) col (int) — столбец (0-79) symbol (char) — выводимый символ	void	Записывает один символ с заданным цветом в видеопамять по адресу 0xB8000 для отображения на экране.
screen_print_string	color (int) — цвет текста text (const char*) — указатель на строку line (unsigned int) — номер строки экрана	void	Выводит строку текста целиком на указанную строку экрана, используя прямой доступ к видеопамяти.
set_cursor	row (unsigned	void	Перемещает

	int) — строка col (unsigned int) — столбец		аппаратный курсор текстового режима путем записи в порты VGA 0x3D4 и 0x3D5.
erase_display	void	void	Очищает весь экран, заполняя видеопамять пробелами с текущим цветом и сбрасывает позицию курсора.
show_prompt	void	void	Выводит приглашение командной строки "# " и устанавливает курсор в позицию после него.
РАБОТА С ПРЕРЫВАНИЯМИ			
port_in_byte	port (unsigned short) — номер порта ввода-вывода	unsigned char — прочитанный байт	Читает байт из указанного порта (ассемблерная вставка inb). Используется для получения скан-кода клавиатуры из порта 0x60.

port_out_byte	port (unsigned short) — номер порта value (unsigned char) — значение для записи	void	Записывает байт в указанный порт (ассемблерная вставка outb). Используется для отправки сигнала "конец прерывания" в PIC и управления курсором.
register_interrupt	vector (int) — номер вектора прерывания selector (unsigned short) — селектор сегмента кода flags (unsigned char) — атрибуты шлюза handler (InterruptRoutine) — указатель на функцию-обработчик	void	Заполняет запись в таблице дескрипторов прерываний (IDT) для заданного вектора, связывая номер прерывания с конкретным обработчиком.
initialize_interrupts	void	void	Инициализирует все 256 записей IDT обработчиком по умолчанию

			(default_handler).
load_interrupts	void	void	Загружает подготовленную таблицу IDT в процессор с помощью инструкции lidt.
setup_keyboard	void	void	Регистрирует keyboard_handler на вектор 0x09 (прерывание клавиатуры) и разрешает его в контроллере прерываний (PIC).
keyboard_handler	(вызывается аппаратно)	void (инструкция iret)	Обработчик прерываний клавиатуры. Читает скан-код из порта 0x60, обрабатывает нажатия Shift, преобразует скан-код в ASCII-символ, заносит символ в буфер и выводит на экран. Поддерживает

			Backspace и Enter.
СТРОКОВЫЕ ФУНКЦИИ			
compare_strings	str1 (const char*) — первая строка str2 (const char*) — вторая строка	int (0 — строки равны)	Сравнивает две строки до первого различия или конца строки. Используется для распознавания команд info и shutdown.
compare_partial	str1 (const char*) — строка str2 (const char*) — шаблон length (int) — количество символов	int (0 — первые length символов равны)	Сравнивает первые length символов строки с шаблоном. Используется для распознавания команды expr (первые 4 символа должны быть "expr").
КАЛЬКУЛЯТОР(КОМАНДА EXPR)			
evaluate_math_expression	expr (const char*) — строка с арифметическим выражением result (int*) — указатель для сохранения	int (код ошибки: 0 — успех 2 — неверное выражение 3 — переполнение	**Основная функция калькулятора.** Разбирает арифметическое выражение, поддерживает

	результата	4 — деление на 0)	унарные операторы (+ и -), учитывает приоритет операций (* и / выше, чем + и -). Использует два стека (чисел и операторов). При вычислениях проверяет целочисленное переполнение и деление на ноль.
perform_operation	op (char) — оператор (+, -, *, /) left (int) — левый операнд right (int) — правый операнд error_code (int*) — указатель для кода ошибки	int (результат операции)	Выполняет одну арифметическую операцию, вызывая соответствующие функции безопасного вычисления (safe_add, safe_sub, safe_mul, safe_div).
push_number / pop_number	val (int) — значение для стека чисел	void / int	Работа со стеком чисел калькулятора.
push_operator /	op (char) —	void / char	Работа со стеком

pop_operator	оператор для стека операторов		операторов калькулятора.
render_number	number (int) — число для вывода	void	Преобразует целое число в строку и выводит на экран. Корректно обрабатывает крайний случай -2147483648 (минимальное значение int).
КОМАНДЫ ОС			
execute_info	void	void	Реализует команду info. Выводит информацию об авторе, группе, использованных компиляторах (GNU as, gcc) и цвете, выбранном пользователем в загрузчике (считанном из адреса 0x8000).
execute_expression	command_line (const char*) — полная строка, введенная	void	Реализует команду expr. Извлекает из строки

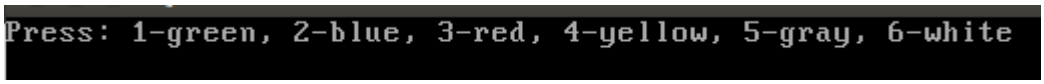
	пользователем		выражение (отсекая "expr" и пробелы), вызывает evaluate_math_expression и выводит результат или сообщение об ошибке.
execute_shutdown	void	void	Реализует команду shutdown. Выводит сообщение о выключении и записывает значение 0x2000 в порт 0x604, что вызывает завершение работы в эмуляторе QEMU.
ТОЧКА ВХОДА			
kmain	void	int	Главная функция ядра. Считывает цвет из 0x8000, инициализирует экран,

			настраивает прерывания, запускает обработчик клавиатуры и входит в бесконечный цикл обработки команд.

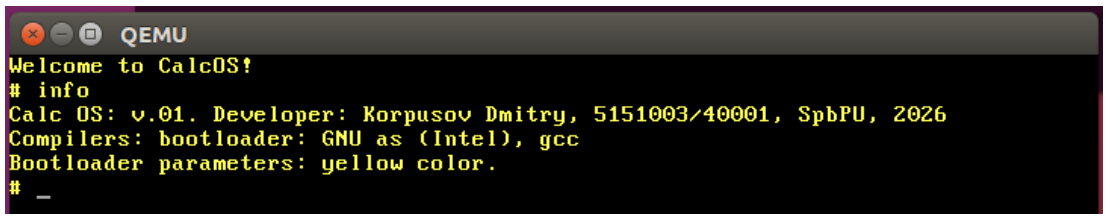
Таблица 1 «Функции kernel.cpp»

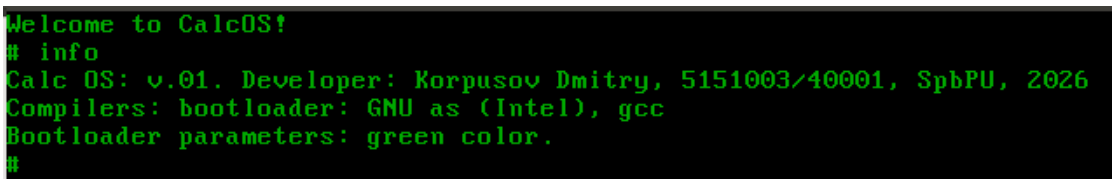
ТЕСТИРОВАНИЕ ЯДРА CalcOS

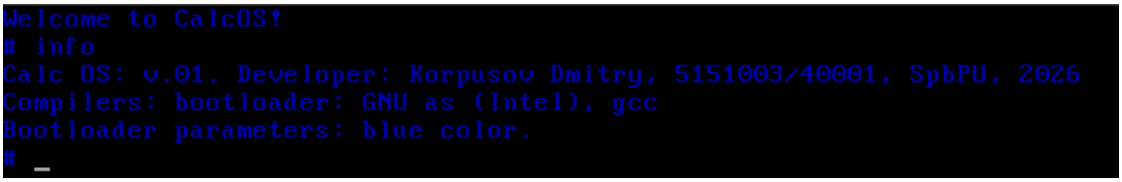
Для проверки корректности работы разработанной операционной системы CalcOS было проведено тестирование, включающее 15 различных сценариев. Тесты охватывают все реализованные команды (info, expr, shutdown), проверку обработки ошибок, граничные случаи и работу пользовательского ввода.

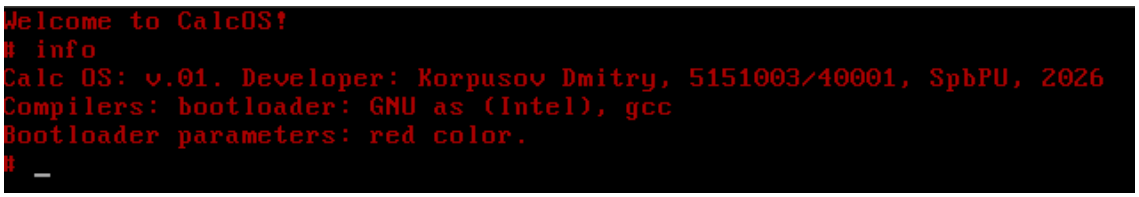


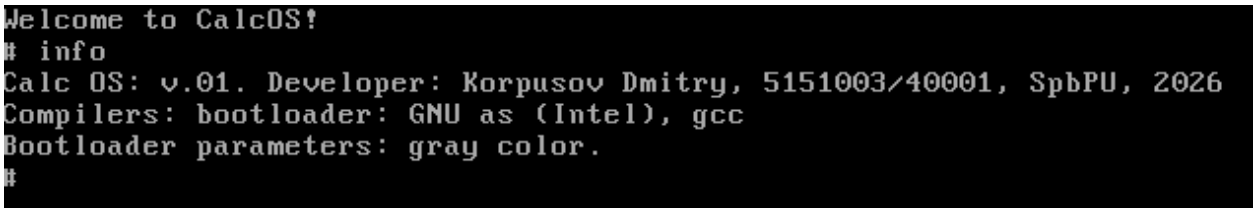
Команда info:





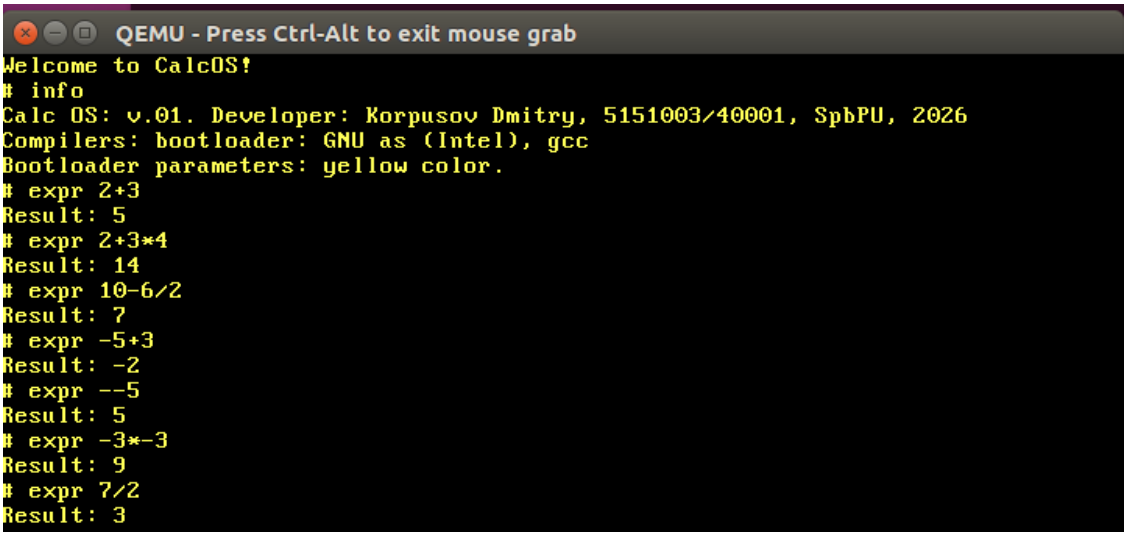






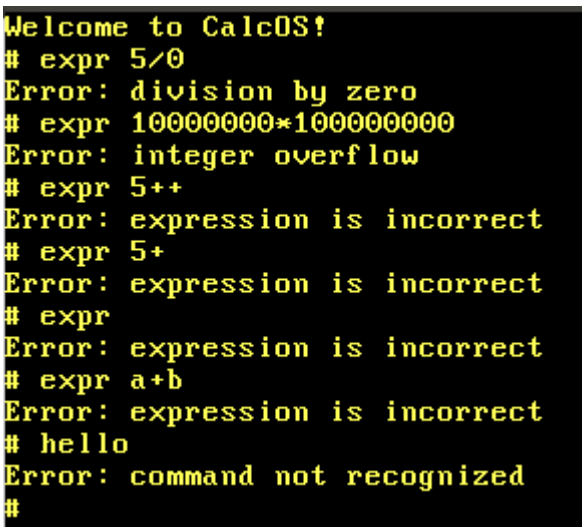


Команды expr (корректные выражения):





Команды expr (некорректные выражения):



ВЫВОД

В ходе выполнения лабораторной работы была разработана минимальная операционная система CalcOS, реализующая функционал простого калькулятора в защищенном режиме процессора x86. В процессе работы были изучены теоретические основы функционирования операционных систем: процесс загрузки компьютера, роль BIOS, структура загрузочного сектора, принципы работы процессора в реальном и защищенном режимах, механизм прерываний и система защиты памяти. Был разработан загрузчик на языке ассемблера (GNU as, Intel синтаксис), который обеспечивает взаимодействие с пользователем через BIOS для выбора цвета текста, сохраняет этот параметр в память, загружает ядро с диска с использованием прерывания int 0x13, переключает процессор в защищенный 32-битный режим и передает управление ядру. Ядро, реализованное на языке C++ в среде без стандартной библиотеки, считывает переданный загрузчиком параметр цвета, настраивает таблицу дескрипторов прерываний (IDT) и обрабатывает прерывания клавиатуры, обеспечивая интерактивный ввод команд с поддержкой Backspace и ограничением длины строки. В ядре реализованы команды согласно варианту задания: info (вывод информации об авторе и параметрах загрузки), shutdown (выключение компьютера) и expr (вычисление арифметических выражений). Особое внимание было уделено

реализации калькулятора, поддерживающего бинарные операции с учетом их приоритета, унарные операторы, а также корректную обработку ошибок: некорректного выражения, переполнения и деления на ноль. Проведенное тестирование (15 тестов) подтвердило корректность работы всех компонентов системы — загрузчик правильно сохраняет и передает параметры, ядро корректно обрабатывает пользовательский ввод, а калькулятор вычисляет выражения в соответствии с математическими правилами и адекватно реагирует на ошибочные ситуации. Таким образом, цель работы достигнута, все задачи выполнены в полном соответствии с вариантом задания.

Листинг А «bootsect.asm»

```
;
; Загрузчик ОС CalcOS
; Компиляция: as --32 -o bootsect.o bootsect.asm
;              ld -Ttext 0x7c00 --oformat binary -m elf_i386 -o
bootsect.bin bootsect.o
;

.intel_syntax noprefix
.code16

_start:
    # Инициализация сегментов
    mov ax, cs
    mov ds, ax
    mov ss, ax
    mov sp, 0x7C00

    # Очистка экрана (установка видеорежима 80x25)
    mov al, 0x03
    mov ah, 0x00
    int 0x10

    # Вывод сообщения с меню выбора цвета
```

```
    mov bx, offset loading_str
    call puts

get_choice:
    # Ожидание нажатия клавиши
    xor ah, ah
    int 0x16

    # Проверка, что нажата цифра от 1 до 6
    cmp al, '1'
    jb get_choice
    cmp al, '6'
    ja get_choice

    # Преобразование цифры в код цвета
    sub al, '1'
    mov bx, offset color_table
    xlat

    # Сохранение кода цвета по адресу 0x8000
    push ax
    xor ax, ax
    mov es, ax
    pop ax
    mov es:[0x8000], al

    # Очистка экрана перед загрузкой
    mov al, 0x03
    mov ah, 0x00
    int 0x10

    # Загрузка ядра со второй дискеты (fd1)
    # Адрес загрузки: 0x1000:0x0000 (физический 0x10000)
    mov bx, 0x1000
    mov es, bx
    xor bx, bx
```

```
    mov dl, 0x01      # Номер диска: 1 - второй дисковод
    mov dh, 0x00      # Номер головки: 0
    mov ch, 0x00      # Номер цилиндра: 0
    mov cl, 0x01      # Номер сектора: 1 (нумерация с 1)
    mov al, 0x1A      # Количество секторов: 26
    mov ah, 0x02      # Функция чтения секторов
    int 0x13          # Вызов BIOS

    # Переход в защищенный режим
    cli               # Отключение прерываний

    # Включение адресной линии A20
    in al, 0x92
    or al, 2
    out 0x92, al

    # Загрузка глобальной таблицы дескрипторов
    lgdt gdt_info

    # Установка бита PE (Protection Enable) в CR0
    mov eax, cr0
    or al, 1
    mov cr0, eax

    # Дальний прыжок для сброса конвейера и загрузки CS
    jmp 0x8:protected_mode


# Функция вывода строки через BIOS
puts:
    push bx
    mov ah, 0x0E
.loop:
    mov al, [bx]
    test al, al
    jz end_puts
```

```
    int 0x10
    inc bx
    jmp .loop
end_puts:
    pop bx
    ret

# Строка меню
loading_str:
    .asciz "Press: 1-green, 2-blue, 3-red, 4-yellow, 5-gray, 6-
white\r\n"

# Таблица соответствия цифр кодам цвета VGA
color_table:
    .byte 0x02, 0x01, 0x04, 0x0E, 0x07, 0x0F
    #    green  blue  red  yellow gray  white

.code32
protected_mode:
    # Настройка сегментов данных
    mov ax, 0x10
    mov ds, ax
    mov es, ax
    mov ss, ax
    mov esp, 0x90000   # Установка стека

    # Передача управления ядру
    call 0x10000

# Глобальная таблица дескрипторов (GDT)
gdt:
    .byte 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00   #
Нулевой дескриптор
    .byte 0xff, 0xff, 0x00, 0x00, 0x00, 0x9A, 0xCF, 0x00   #
Сегмент кода (селектор 0x08)
```

```
    .byte 0xff, 0xff, 0x00, 0x00, 0x00, 0x92, 0xCF, 0x00   #
Сегмент данных (селектор 0x10)
gdt_end:

# Информация о GDT для команды lgdt
gdt_info:
    .word gdt_end - gdt - 1        # Размер GDT минус 1
    .word gdt, 0                   # 32-битный адрес GDT

# Заполнение до 512 байт и загрузочная сигнатура
.zero 512-($ - _start) - 2
.word 0xAA55
```

Листин Б «kernel.cpp»

```
// Ядро ОС CalcOS
// Компиляция: g++ -ffreestanding -m32 -o kernel.o -c kernel.cpp
//             ld --oformat binary -Ttext 0x10000 -o kernel.bin -
e kmain -m elf_i386 kernel.o

__asm("jmp kmain");

//=============================================================
===============
//                            СИСТЕМНЫЕ КОНСТАНТЫ
//=============================================================
===============

#define GATE_TYPE 0x0E              // Тип шлюза прерывания (32-
битный)
#define CODE_SELECTOR 0x08        // Селектор сегмента кода
#define PIC_BASE 0x20               // Базовый порт контроллера
прерываний
#define CRT_CONTROL 0x3D4         // Порт управления курсором
#define POWER_PORT 0x604          // Порт для выключения (QEMU)
#define VIDEO_AREA 0xb8000        // Адрес видеопамяти
#define SCREEN_WIDTH 80            // Ширина экрана
```

```
#define MAX_ROWS 25                  // Высота экрана
#define COMMAND_SIZE_LIMIT 40        // Максимальная длина команды
#define BOOT_COLOR_LOCATION ((volatile unsigned char*)0x8000) //
Адрес цвета от загрузчика
#define MAX_INT 2147483647           // Максимальное значение int
#define MIN_INT (-2147483647-1)      // Минимальное значение int
#define  STACK_DEPTH  100                      //  Глубина  стеков
калькулятора

//=============================================================
===============
//                            СТРУКТУРЫ ДАННЫХ
//=============================================================
===============

// Дескриптор прерывания (формат IDT)
struct InterruptDescriptor {
    unsigned  short  offset_low;         //  Младшие  биты  адреса
обработчика
    unsigned short segment;          // Селектор сегмента кода
    unsigned char reserved;          // Зарезервировано (всегда 0)
    unsigned  char  attributes;         // Атрибуты шлюза (P, DPL,
тип)
    unsigned  short  offset_high;        //  Старшие  биты  адреса
обработчика
} __attribute__((packed));

// Указатель на IDT для команды lidt
struct DescriptorPointer {
    unsigned short table_size;       // Размер таблицы минус 1
    unsigned int table_base;         // Базовый адрес таблицы
} __attribute__((packed));

typedef void (*InterruptRoutine)();
```

```
//=============================================================
===============
//                              ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ
//=============================================================
===============

static struct InterruptDescriptor idt_entries[256];   // Таблица
IDT
static struct DescriptorPointer idt_ptr;                      //
Указатель на IDT

static unsigned char current_color = 0x07;              // Текущий
цвет текста
static int cursor_row = 1;                              // Текущая
строка курсора
static int cursor_column = 0;                                 //
Текущий столбец курсора

static char input_buffer[COMMAND_SIZE_LIMIT];            // Буфер
ввода команды
static int buffer_position = 0;                               //
Текущая позиция в буфере
static int input_enabled = 0;                                 //
Разрешен ли ввод
static int command_available = 0;                          // Флаг
готовности команды

//=============================================================
===============
//                              ПРОТОТИПЫ
//=============================================================
===============

static inline void port_out_byte(unsigned short port, unsigned
char value);
```

```
static void port_out_word(unsigned short port, unsigned short
value);
static inline unsigned char port_in_byte(unsigned short port);
static void screen_put_char(unsigned char color, int row, int
col, char symbol);
static void screen_print_string(int color, const char* text,
unsigned int line);
static void set_cursor(unsigned int row, unsigned int col);
static void erase_display();
static void verify_display_space();
static void advance_to_next_line();
static void show_prompt();
static void register_interrupt(int vector, unsigned short
selector, unsigned char flags, InterruptRoutine handler);
static void initialize_interrupts();
static void load_interrupts();
static inline void allow_interrupts();
static inline void block_interrupts();
static void setup_keyboard();
extern "C" void default_handler();
extern "C" void keyboard_handler();
static int compare_strings(const char* str1, const char* str2);
static int compare_partial(const char* str1, const char* str2,
int length);
static void execute_info();
static void execute_expression(const char* command_line);
static void execute_shutdown();

//================================================================
===============
//                          КАЛЬКУЛЯТОР
//================================================================
===============

static int number_stack[STACK_DEPTH];      // Стек для чисел
```

```
static int number_top = -1;                    // Вершина стека
чисел

static  int  operator_stack[STACK_DEPTH];        //  Стек  для
операторов
static int operator_top = -1;                  // Вершина стека
операторов

static int calc_error = 0;                    // Флаг ошибки

// Инициализация калькулятора
static void initialize_engine() {
    number_top = -1;
    operator_top = -1;
    calc_error = 0;
}

// Работа со стеком чисел
static void push_number(int val) {
    if (number_top < STACK_DEPTH - 1) {
        number_stack[++number_top] = val;
    }
}

static int pop_number() {
    if (number_top >= 0) {
        return number_stack[number_top--];
    }
    return 0;
}

static int peek_number() {
    if (number_top >= 0) {
        return number_stack[number_top];
    }
    return 0;
```

```
}

// Работа со стеком операторов
static void push_operator(char op) {
    if (operator_top < STACK_DEPTH - 1) {
        operator_stack[++operator_top] = (int)op;
    }
}

static char pop_operator() {
    if (operator_top >= 0) {
        return (char)operator_stack[operator_top--];
    }
    return 0;
}

static char peek_operator() {
    if (operator_top >= 0) {
        return (char)operator_stack[operator_top];
    }
    return 0;
}

static int is_operator_on_top() {
    if (operator_top >= 0) {
        char op = (char)operator_stack[operator_top];
        return (op == '+' || op == '-' || op == '*' || op ==
'/') ? 1 : 0;
    }
    return 0;
}

// Матрица приоритетов операций
static const signed char precedence_grid[4][4] = {
    { 0, 0, -1, -1 }, // '+' по отношению к +,-,*,/
    { 0, 0, -1, -1 }, // '-' по отношению к +,-,*,/
```

```
    { 1, 1, 0, 0  },  // '*' по отношению к +,-,*,/
    { 1, 1, 0, 0  }   // '/' по отношению к +,-,*,/
};

// Получение кода операции для матрицы приоритетов
static int get_operation_code(char op) {
    if (op == '+') return 0;
    if (op == '-') return 1;
    if (op == '*') return 2;
    if (op == '/') return 3;
    return -1;
}

// Проверка приоритета: имеет ли first приоритет над second
static int has_priority(char first, char second) {
    int idx1 = get_operation_code(first);
    int idx2 = get_operation_code(second);
    if (idx1 < 0 || idx2 < 0) return 0;
    return precedence_grid[idx1][idx2] > 0;
}

// Безопасное сложение с проверкой переполнения
static int safe_add(int x, int y, int* fault) {
    *fault = 0;
    if (y > 0 && x > MAX_INT - y) { *fault = 1; return 0; }
    if (y < 0 && x < MIN_INT - y) { *fault = 1; return 0; }
    return x + y;
}

// Безопасное вычитание с проверкой переполнения
static int safe_sub(int x, int y, int* fault) {
    *fault = 0;
    if (y < 0 && x > MAX_INT + y) { *fault = 1; return 0; }
    if (y > 0 && x < MIN_INT + y) { *fault = 1; return 0; }
    return x - y;
}
```

```
// Безопасное умножение с проверкой переполнения
static int safe_mul(int x, int y, int* fault) {
    *fault = 0;

    if (x == 0 || y == 0) return 0;

    // Особый случай: MIN_INT * -1 = переполнение
    if (x == MIN_INT && y == -1) { *fault = 1; return 0; }
    if (y == MIN_INT && x == -1) { *fault = 1; return 0; }

    if (x > 0) {
        if (y > 0) { if (x > MAX_INT / y) { *fault = 1; return
0; } }
        else { if (y < MIN_INT / x) { *fault = 1; return 0; } }
    } else {
        if (y > 0) { if (x < MIN_INT / y) { *fault = 1; return
0; } }
        else { if (x < MAX_INT / y) { *fault = 1; return 0; } }
    }

    return x * y;
}

// Безопасное деление с проверкой деления на ноль и переполнения
static int safe_div(int x, int y, int* fault) {
    *fault = 0;

    if (y == 0) {
        *fault = 2;  // Код ошибки: деление на ноль
        return 0;
    }

    // Особый случай: MIN_INT / -1 = переполнение
    if (x == MIN_INT && y == -1) {
        *fault = 1;
```

```
        return 0;
    }

    return x / y;
}

// Выполнение одной операции с проверкой ошибок
static int perform_operation(char operation, int left, int
right, int* error_code) {
    int fault = 0;
    int outcome = 0;

    if (operation == '+') outcome = safe_add(left, right,
&fault);
    else if (operation == '-') outcome = safe_sub(left, right,
&fault);
    else if (operation == '*') outcome = safe_mul(left, right,
&fault);
    else if (operation == '/') outcome = safe_div(left, right,
&fault);

    if (fault == 1) *error_code = 3;  // Переполнение
    else if (fault == 2) *error_code = 4;  // Деление на ноль

    return outcome;
}

// Извлечение числа из строки с проверкой переполнения
static int fetch_numeric(const char* source, int* position, int*
output) {
    int start = *position;
    int current = start;
    int sign = 1;
    *output = 0;

    // Пропуск пробелов
```

```
    while (source[current] == ' ') current++;

    // Обработка знака
    if (source[current] == '-') {
        sign = -1;
        current++;
    } else if (source[current] == '+') {
        current++;
    }

    // Проверка, что следующий символ - цифра
    if (source[current] < '0' || source[current] > '9') {
        return -1;  // Не число
    }

    unsigned int value = 0;
    int digits = 0;

    // Преобразование строки в число с проверкой переполнения
    while (source[current] >= '0' && source[current] <= '9') {
        unsigned int digit = (unsigned int)(source[current] -
'0');

        // Проверка переполнения для знакового int
        unsigned int limit = (sign == -1) ? 2147483648u :
2147483647u;

        if (value > limit / 10u || (value == limit / 10u &&
digit > limit % 10u)) {
            return -2;  // Переполнение
        }

        value = value * 10u + digit;
        current++;
        digits++;
    }
```

```
    if (digits == 0) {
        return -1;  // Не число
    }

    // Применение знака с учетом специального случая MIN_INT
    if (sign == -1) {
        if (value == 2147483648u) {
            *output = MIN_INT;
        } else {
            *output = -(int)value;
        }
    } else {
        *output = (int)value;
    }

    *position = current;
    return 0;  // Успех
}

// Основная функция вычисления арифметического выражения
static int evaluate_math_expression(const char* expr, int*
result) {
    initialize_engine();

    int pos = 0;
    int expect_operand = 1;    // Ожидается операнд (начало
выражения или после оператора)
    int unary = 1;            // Накопление унарных знаков
    int error = 0;

    while (expr[pos] != '\0' && expr[pos] != '\n') {
        // Пропуск пробелов
        while (expr[pos] == ' ') pos++;
        if (expr[pos] == '\0') break;
```

```
        if (expect_operand) {
            // Обработка унарных операторов
            if (expr[pos] == '+' || expr[pos] == '-') {
                if (expr[pos] == '-') unary *= -1;
                pos++;
                continue;
            }

            // Обработка числа
            if (expr[pos] >= '0' && expr[pos] <= '9') {
                int num;
                int status = fetch_numeric(expr, &pos, &num);

                if (status == -2) return 3;   // Переполнение
числа
                if (status < 0) return 2;         // Неверное
выражение

                num *= unary;  // Применение унарных знаков
                unary = 1;      // Сброс унарных знаков

                push_number(num);
                expect_operand = 0;     // Теперь ожидается
оператор
            } else {
                return 2;  // Неверное выражение
            }
        } else {
            // Обработка бинарного оператора
            if (expr[pos] == '+' || expr[pos] == '-' ||
                expr[pos] == '*' || expr[pos] == '/') {

                char op = expr[pos];
                pos++;
```

```
                // Выполнение операций с более высоким
приоритетом
                while (operator_top >= 0) {
                    char top = peek_operator();

                    if (has_priority(top, op)) {
                        pop_operator();
                        int b = pop_number();
                        int a = pop_number();

                        int res = perform_operation(top, a, b,
&error);
                        if (error) return error;

                        push_number(res);
                    } else {
                        break;
                    }
                }

                push_operator(op);
                expect_operand = 1;     // После оператора
ожидается операнд
            } else {
                return 2;  // Неверное выражение
            }
        }
    }

    // Выполнение оставшихся операций
    while (operator_top >= 0) {
        char op = pop_operator();
        int b = pop_number();
        int a = pop_number();

        int res = perform_operation(op, a, b, &error);
```

```
        if (error) return error;

        push_number(res);
    }

    // Проверка результата
    if (number_top != 0) {
        return 2;  // Неверное выражение
    }

    *result = number_stack[0];
    return 0;  // Успех
}

// Вывод числа на экран
static void render_number(int number) {
    // Особый случай: минимальное значение int
    if (number == MIN_INT) {
        const char* literal = "-2147483648";
        while (*literal) {
            screen_put_char(current_color,           cursor_row,
cursor_column, *literal++);
            cursor_column++;
        }
        return;
    }

    char buffer[12];
    int pos = 0;
    int is_negative_flag = 0;

    if (number == 0) {
        screen_put_char(current_color,               cursor_row,
cursor_column, '0');
        cursor_column++;
        return;
```

```
    }

    if (number < 0) {
        is_negative_flag = 1;
        number = -number;
    }

    // Преобразование числа в строку (цифры заносятся в обратном
порядке)
    while (number > 0) {
        buffer[pos++] = '0' + (number % 10);
        number /= 10;
    }

    // Вывод знака минус, если нужно
    if (is_negative_flag) {
        screen_put_char(current_color,                cursor_row,
cursor_column, '-');
        cursor_column++;
    }

    // Вывод цифр в правильном порядке
    while (pos > 0) {
        pos--;
        screen_put_char(current_color,                cursor_row,
cursor_column, buffer[pos]);
        cursor_column++;
    }
}

//================================================================
===============
//                          БАЗОВЫЙ ВВОД-ВЫВОД
//================================================================
===============
```

```
// Запись байта в порт
static inline void port_out_byte(unsigned short port, unsigned
char value)
{
    asm volatile ("outb %0, %1" : : "a"(value), "Nd"(port));
}

// Запись слова (2 байт) в порт
static void port_out_word(unsigned short port, unsigned short
value)
{
    asm volatile ("outw %0, %1" : : "a"(value), "Nd"(port));
}

// Чтение байта из порта
static inline unsigned char port_in_byte(unsigned short port)
{
    unsigned char result;
    asm volatile ("inb %1, %0" : "=a"(result) : "Nd"(port));
    return result;
}

// Вывод одного символа на экран через видеопамять
static void screen_put_char(unsigned char color, int row, int
col, char symbol)
{
    unsigned char* video_segment = (unsigned char*)VIDEO_AREA;
    video_segment += (row * SCREEN_WIDTH + col) * 2;
    video_segment[0] = (unsigned char)symbol;
    video_segment[1] = color;
}

// Вывод строки на экран
void screen_print_string(int color, const char* text, unsigned
int line)
{
```

```
    unsigned char* video_pointer = (unsigned char*)VIDEO_AREA;
    video_pointer += SCREEN_WIDTH * 2 * line;

    while (*text) {
        video_pointer[0] = (unsigned char)*text;
        video_pointer[1] = color;
        video_pointer += 2;
        text++;
    }
}

// Установка позиции аппаратного курсора
static void set_cursor(unsigned int row, unsigned int col)
{
    unsigned short linear_position = (row * SCREEN_WIDTH) + col;

    port_out_byte(CRT_CONTROL, 0x0F);
    port_out_byte(CRT_CONTROL + 1, (unsigned
char)(linear_position & 0xFF));

    port_out_byte(CRT_CONTROL, 0x0E);
    port_out_byte(CRT_CONTROL + 1, (unsigned
char)((linear_position >> 8) & 0xFF));
}

//=============================================================
===============
//                          УПРАВЛЕНИЕ ЭКРАНОМ
//=============================================================
===============

// Очистка всего экрана
static void erase_display()
{
    unsigned char* video_region = (unsigned char*)VIDEO_AREA;
```

```
    for (int line = 0; line < MAX_ROWS; line++) {
        for (int column = 0; column < SCREEN_WIDTH; column++) {
            video_region[(line * SCREEN_WIDTH + column) * 2] = '
';
            video_region[(line * SCREEN_WIDTH + column) * 2 + 1]
= current_color;
        }
    }

    cursor_row = 0;
    cursor_column = 0;
    set_cursor(cursor_row, cursor_column);
}

// Проверка, не заполнен ли экран
static void verify_display_space()
{
    if (cursor_row >= MAX_ROWS - 1) {
        erase_display();
        screen_print_string(current_color, "Welcome to CalcOS!",
cursor_row);
        cursor_row++;
        show_prompt();
    }
}

// Перевод на новую строку
static void advance_to_next_line()
{
    cursor_row++;
    cursor_column = 0;
    set_cursor(cursor_row, cursor_column);
    verify_display_space();
}

// Вывод приглашения командной строки
```

```
static void show_prompt()
{
    screen_print_string(current_color, "# ", cursor_row);
    cursor_column = 2;
    set_cursor(cursor_row, cursor_column);
}

//=============================================================
===============
//                          ПРЕРЫВАНИЯ
//=============================================================
===============

// Регистрация обработчика прерывания в IDT
static  void  register_interrupt(int  vector,  unsigned  short
selector,
                                 unsigned       char       flags,
InterruptRoutine handler)
{
    unsigned int routine_address = (unsigned int)handler;
    idt_entries[vector].offset_low          =          (unsigned
short)(routine_address & 0xFFFF);
    idt_entries[vector].segment = selector;
    idt_entries[vector].reserved = 0;
    idt_entries[vector].attributes = flags;
    idt_entries[vector].offset_high          =          (unsigned
short)(routine_address >> 16);
}

// Инициализация IDT обработчиками по умолчанию
static void initialize_interrupts()
{
    int      total_entries      =      sizeof(idt_entries)      /
sizeof(idt_entries[0]);
```

```
    for (int vector_idx = 0; vector_idx < total_entries;
vector_idx++) {
        register_interrupt(vector_idx, CODE_SELECTOR, 0x80 |
GATE_TYPE, default_handler);
    }
}

// Загрузка IDT в процессор
static void load_interrupts()
{
    int total_entries = sizeof(idt_entries) /
sizeof(idt_entries[0]);

    idt_ptr.table_base = (unsigned int)&idt_entries[0];
    idt_ptr.table_size = (sizeof(struct InterruptDescriptor) *
total_entries) - 1;
    asm volatile("lidt %0" : : "m"(idt_ptr));
}

// Разрешение прерываний
static inline void allow_interrupts() { asm volatile("sti"); }

// Запрещение прерываний
static inline void block_interrupts() { asm volatile("cli"); }

//================================================================
===============
//                          КЛАВИАТУРА
//================================================================
===============

// Настройка обработчика клавиатуры
static void setup_keyboard()
{
    register_interrupt(0x09, CODE_SELECTOR, 0x80 | GATE_TYPE,
keyboard_handler);
```

```
    // Разрешение только прерывания клавиатуры (IRQ1, бит 0x02)
    port_out_byte(PIC_BASE + 1, 0xFF ^ 0x02);
}

//=================================================================
===============
//                          ОБРАБОТЧИКИ
//=================================================================
===============

// Обработчик прерываний по умолчанию
extern "C" void default_handler()
{
    asm volatile("pusha");
    asm volatile("popa; leave; iret");
}

// Обработчик прерываний клавиатуры
extern "C" void keyboard_handler()
{
    asm volatile("pusha");

    static int shift_pressed = 0;  // Состояние клавиши Shift

    // Проверка, есть ли данные от клавиатуры
    if (port_in_byte(0x64) & 0x01) {
        unsigned char scan_code = port_in_byte(0x60);

        // Обработка нажатия Shift
        if (scan_code == 0x2A || scan_code == 0x36) {
            shift_pressed = 1;
        }
        // Обработка отпускания Shift
        else if (scan_code == 0xAA || scan_code == 0xB6) {
            shift_pressed = 0;
        }
```

```
        // Обработка обычных клавиш (скан-код < 128 - нажатие)
        else if (scan_code < 128) {

            // Таблица раскладки без Shift
            static const char unshifted_layout[128] = {
                0,0,'1','2','3','4','5','6','7','8','9','0','-
','=', '\b','\t',

'q','w','e','r','t','y','u','i','o','p','[',']','\n',0,

'a','s','d','f','g','h','j','k','l',';','\'','`',0,'\\',

'z','x','c','v','b','n','m',',','.','/',0,'*',0,' ',0,
            };

            // Таблица раскладки с Shift
            static const char shifted_layout[128] = {

0,0,'!','@','#','$','%','^','&','*','(',')','_','+', '\b','\t',

'Q','W','E','R','T','Y','U','I','O','P','{','}','\n',0,

'A','S','D','F','G','H','J','K','L',':','"','~',0,'|',

'Z','X','C','V','B','N','M','<','>','?',0,'*',0,' ',0,
            };

            // Выбор символа в зависимости от Shift
            char      character      =      shift_pressed      ?
shifted_layout[scan_code] : unshifted_layout[scan_code];

            // Обработка символа, если ввод разрешен
            if (input_enabled) {
                if (character == '\n') {
                    // Enter: завершение команды
                    input_buffer[buffer_position] = '\0';
```

```
                command_available = 1;
            }
            else if (character == '\b') {
                // Backspace: удаление последнего символа
                if (buffer_position > 0) {
                    buffer_position--;
                    cursor_column--;
                    screen_put_char(current_color,
cursor_row, cursor_column, ' ');
                    set_cursor(cursor_row, cursor_column);
                }
            }
            else if (character != 0 && buffer_position <
COMMAND_SIZE_LIMIT - 1) {
                // Обычный символ: добавление в буфер и на
экран
                input_buffer[buffer_position++] = character;
                screen_put_char(current_color,    cursor_row,
cursor_column, character);
                cursor_column++;
                set_cursor(cursor_row, cursor_column);
            }
        }
    }

    // Отправка сигнала "конец прерывания" контроллеру PIC
    port_out_byte(PIC_BASE, 0x20);
    asm volatile("popa; leave; iret");
}

//============================================================
===============
//                          СТРОКОВЫЕ ФУНКЦИИ
//============================================================
===============
```

```
// Сравнение двух строк
static int compare_strings(const char* str1, const char* str2)
{
    while (*str1 && *str1 == *str2) {
        str1++;
        str2++;
    }
    return *(unsigned char*)str1 - *(unsigned char*)str2;
}

// Сравнение первых length символов строк
static int compare_partial(const char* str1, const char* str2,
int length)
{
    int offset = 0;
    while (offset < length && *str1 && *str1 == *str2) {
        str1++;
        str2++;
        offset++;
    }

    if (offset == length) return 0;
    return *(unsigned char*)str1 - *(unsigned char*)str2;
}

//================================================================
===============
//                          КОМАНДЫ
//================================================================
===============

// Команда info: вывод информации об ОС
static void execute_info()
{
    advance_to_next_line();
```

```
    screen_print_string(current_color, "Calc OS: v.01.
Developer: Korpusov Dmitry, 5151003/40001, SpbPU, 2026",
cursor_row);
    cursor_row++;
    verify_display_space();

    screen_print_string(current_color, "Compilers: bootloader:
GNU as (Intel), gcc", cursor_row);
    cursor_row++;
    verify_display_space();

    // Преобразование кода цвета в текстовое описание
    const char* color_description;
    switch(current_color) {
        case 0x02: color_description = "green"; break;
        case 0x01: color_description = "blue"; break;
        case 0x04: color_description = "red"; break;
        case 0x0E: color_description = "yellow"; break;
        case 0x07: color_description = "gray"; break;
        case 0x0F: color_description = "white"; break;
        default: color_description = "unknown"; break;
    }

    // Формирование строки с информацией о цвете
    char combined_text[64];
    char* destination = combined_text;
    const char* intro_part = "Bootloader parameters: ";

    while (*intro_part) *destination++ = *intro_part++;
    while (*color_description) *destination++ =
*color_description++;

    const char* ending_part = " color.";
    while (*ending_part) *destination++ = *ending_part++;
    *destination = '\0';
```

```
    screen_print_string(current_color,             combined_text,
cursor_row);
    cursor_row++;
    verify_display_space();

    cursor_column = 0;
    set_cursor(cursor_row, cursor_column);
}

// Команда expr: вычисление арифметического выражения
static void execute_expression(const char* command_line)
{
    advance_to_next_line();

    // Пропуск "expr" и следующих за ним пробелов
    const char* expression_part = command_line + 4;
    while (*expression_part == ' ') expression_part++;

    // Проверка, есть ли выражение
    if (*expression_part == '\0') {
        screen_print_string(current_color, "Error: expression is
incorrect", cursor_row);
        cursor_row++;
        verify_display_space();
        cursor_column = 0;
        set_cursor(cursor_row, cursor_column);
        return;
    }

    // Копирование выражения в отдельный буфер
    char clean_expression[COMMAND_SIZE_LIMIT];
    int i = 0;
    while (expression_part[i] != '\0' && i < COMMAND_SIZE_LIMIT
- 1) {
        clean_expression[i] = expression_part[i];
```

```
        i++;
    }
    clean_expression[i] = '\0';

    // Вычисление выражения
    int computation_result;
    int                    evaluation_status                    =
evaluate_math_expression(clean_expression, &computation_result);

    // Вывод результата или сообщения об ошибке
    if (evaluation_status == 0) {
        screen_print_string(current_color,      "Result:      ",
cursor_row);
        cursor_column = 8;
        render_number(computation_result);
    } else {
        const char* error_message;
        switch(evaluation_status) {
            case 2:
                error_message    =    "Error:    expression    is
incorrect";
                break;
            case 3:
                error_message = "Error: integer overflow";
                break;
            case 4:
                error_message = "Error: division by zero";
                break;
            default:
                error_message = "Error: unknown error";
                break;
        }

        screen_print_string(current_color,        error_message,
cursor_row);
    }
```

```
    cursor_row++;
    verify_display_space();
    cursor_column = 0;
    set_cursor(cursor_row, cursor_column);
}

// Команда shutdown: выключение компьютера
static void execute_shutdown()
{
    screen_print_string(current_color,     "Powering     off...",
cursor_row);
    // Запись в порт выключения QEMU
    port_out_word(0x604, 0x2000);

    // Бесконечный цикл на случай, если выключение не сработало
    while(1) asm("hlt");
}

//=============================================================
================
//                            ТОЧКА ВХОДА
//=============================================================
================

extern "C" int kmain()
{
    // Считывание цвета, переданного загрузчиком
    current_color = *(volatile unsigned char*)0x8000;

    // Вывод приветствия
    screen_print_string(current_color, "Welcome to CalcOS!", 0);
    set_cursor(1, 0);

    // Настройка прерываний
    block_interrupts();
```

```
    initialize_interrupts();
    setup_keyboard();
    load_interrupts();
    allow_interrupts();

    // Запуск интерактивного режима
    input_enabled = 1;
    cursor_row = 1;
    cursor_column = 0;
    show_prompt();

    // Главный цикл обработки команд
    while(1) {
        asm("hlt");  // Ожидание прерывания

        // Если команда готова (нажат Enter)
        if (command_available) {
            block_interrupts();
            command_available = 0;

            // Распознавание команд
            if (compare_strings(input_buffer, "info") == 0) {
                execute_info();
            }
            else  if  (compare_strings(input_buffer,  "shutdown")
== 0) {
                execute_shutdown();
            }
            else if (compare_partial(input_buffer, "expr", 4) ==
0) {
                execute_expression(input_buffer);
            }
            else if (buffer_position > 0) {
                // Неизвестная команда
                advance_to_next_line();
```

```
                screen_print_string(current_color,        "Error:
command not recognized", cursor_row);
                cursor_row++;
                verify_display_space();
            }

            // Очистка буфера ввода
            buffer_position = 0;
            for    (int    clear_idx    =    0;    clear_idx    <
COMMAND_SIZE_LIMIT; clear_idx++) {
                input_buffer[clear_idx] = 0;
            }

            allow_interrupts();

            // Вывод нового приглашения
            cursor_column = 0;
            show_prompt();
            verify_display_space();
        }
    }

    return 0;
}
```